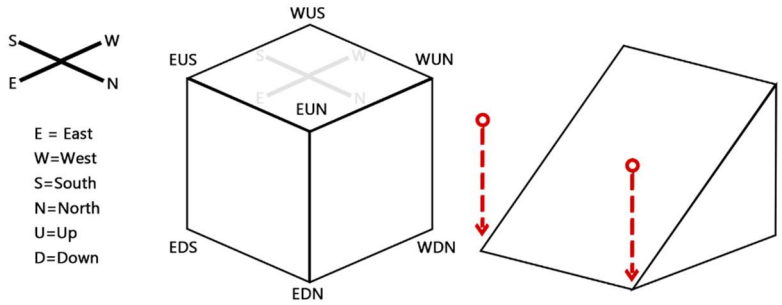
Chapter 3 Study on the Storage Structure of LittleTiles

3.1 Basic Units of LittleTiles

In the LittleTiles mod, a tile is stored as a unit of a Minecraft block and saved in NBT format within the chunk files. A tile is essentially a three-dimensional rectangle, composed of eight vertices:



In the diagram, you can see these eight vertices (except for the ones hidden behind WDS). They are named according to the cardinal directions, with "U" or "D" added to indicate whether the vertex is on the top or bottom.

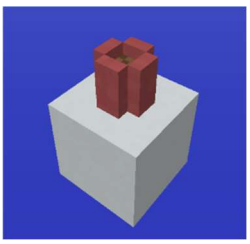
For example, "EUS" represents the southeast direction on the upper side, as shown in the diagram.

Sometimes, you might choose to create a slope, breaking the constraints of Minecraft. When creating a slope, you essentially move some or all of these eight vertices.

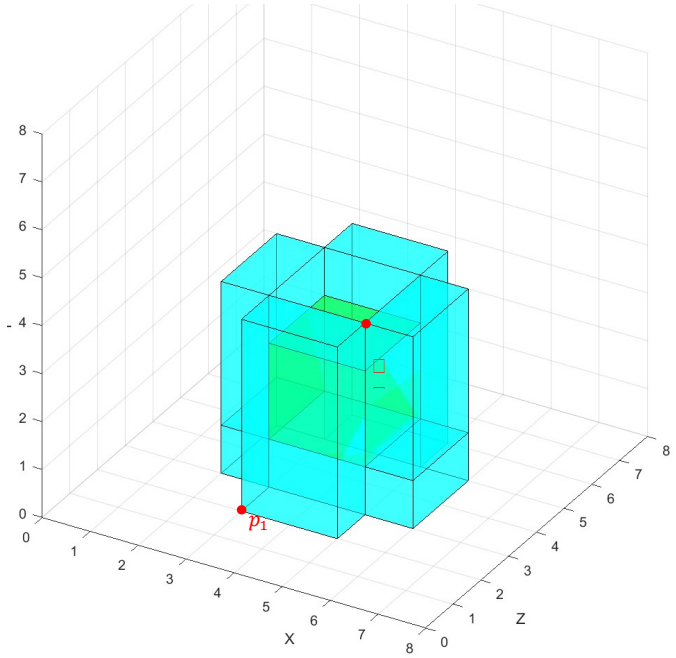
For example, in the shape on the right side of the diagram, the EUS and EUN vertices are simply moved downward along the y-axis until they align with the EDS and EDN corners. The specific offset values are related to voxel and block sizes, which will be discussed further later.

3.2 LittleTiles' Tile

As we learned earlier, tiles are stored within blocks as units and, in turn, stored within chunks. For example, in a block, the storage of tiles can be illustrated with a flower pot example. This example shows a flower pot surrounded by pink ceramic tiles, with soil in the center.



The precision of this storage is 8, as illustrated in the diagram below:



In the left-side diagram, the cyan areas represent the pink ceramic tiles, while the green area in the center represents the soil.

We can clearly see that this three-dimensional image combines multiple tiles within an 8x8x8 (precision) space, ultimately forming the visual we see in the game.

Each tile has two coordinates that define two vertices of the rectangular prism in this three-dimensional space.

For example, the outermost tile has the following coordinates within the block:

$$p_1(3,0,2), p_2(5,4,3)$$

The specific base layer storage of the main data is as follows:

```
Tiles info (BLOCK):
        block coord: 1 5 2
        grid: 8
        id: minecraft:littletilestileentity
        boxes count: 2

      Tile info:
            block id: minecraft:dirt
            boxes:  [3, 1, 3, 5, 3, 5]     // Defines the coordinates of the two points
                                           // of the rectangular prism  [x1, y1, z1, x2, y2, z2]

      Tile info:
            block id: minecraft:stained_hardened_clay:6
            boxes:  [2, 0, 3, 6, 1, 5]
                    [2, 1, 3, 3, 4, 5]
                    [5, 1, 3, 6, 4, 5]
                    [3, 0, 5, 5, 4, 6]    // The tile described in the example
                    [3, 0, 2, 5, 4, 3]
```

3.3 Detailed Explanation of a Tile

3.3.1 Vertex Offset

Previously, we mentioned that a tile has eight vertices. These eight vertices in a block can be determined by two coordinates of the rectangular prism. Thus, to represent the positions of the eight vertices, you only need to know the two coordinates within the block.

Therefore, in the SNBT tags, you will see an array where the first 6 values represent the tile's coordinates within the block (with the exact values determined by the subdivision level (grid) of the block).

$$bBox: [I; x_1, y_1, z_1, x_2, y_2, z_2, \dots]$$

$$\begin{cases} p_1 = (x_1, y_1, z_1) \\ p_2 = (x_2, y_2, z_2) \end{cases}$$

Sometimes, there will also be a series of numbers following these coordinates. These numbers represent the offset data for the eight vertices of the voxel.

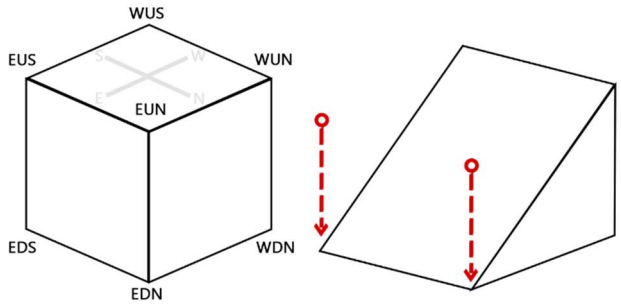
The first number after the coordinates indicates which vertices have been offset and which have not. The subsequent numbers provide the specific offset values in sequence.

Sign bit always negative (1)		Flipped						Vertex																							
								WDS			WDN			WUS			WUN			EDS			EDN			EUS			EUN		
		E	W	S	N	U	D	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X
1	0																										1			1	
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

The diagram lists the function of each bit in this 32-bit signed integer from high to low order. The sign bit is always 1, representing the negative sign. If it were 0, it would indicate a positive number (though LittleTiles does not expect such a situation).

Let's start from the right and move to the left. Every group of three bits represents one vertex, defining which of the three directional axes (x, y, z) the vertex can be offset along. A bit of 1 indicates movement along that axis, while 0 indicates no movement.

Returning to the previous example, assuming the precision is 1, if I offset both the EUS and EUN vertices downward along the y-axis by 1, the result should be the shape shown in the image below:



The bit settings for this image are as follows:

Sign bit always negative (1)		Flipped						Vertex																						
								WDS			WDN			WUS			WUN			EDS			EDN			EUS			EUN		
		E	W	S	N	U	D	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X	Z	**Y**	X	Z	**Y**	X
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	**1**	0	0	**1**	0
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

We can see that the Y values for EUS and EUN are both set to 1, indicating that both vertices have been offset along the y-axis.

This binary sequence converts to a decimal value of -2,147,483,630. This is also why we see many large negative numbers in the string of characters when exporting LittleTiles blueprints.

$$bBox\text{: }[I;0,0,0,1,1,1,-2147483630,\dots]$$

Since we know how to determine which vertices have been offset, the subsequent numbers represent the specific amount of offset for each of those vertices. These values are listed in sequence, so we first ignore the vertices and focus on the x, y, z values. Essentially, these x, y, z values indicate which axes have been offset, so we can also ignore these values.

In total, up to 24 values are needed to represent which of the 24 axes (8 vertices × 3 axes) have been offset. However, storing offsets for all vertices, including those that have not been offset, would be very space-inefficient. Therefore, only the offsets for the moved axes are stored.

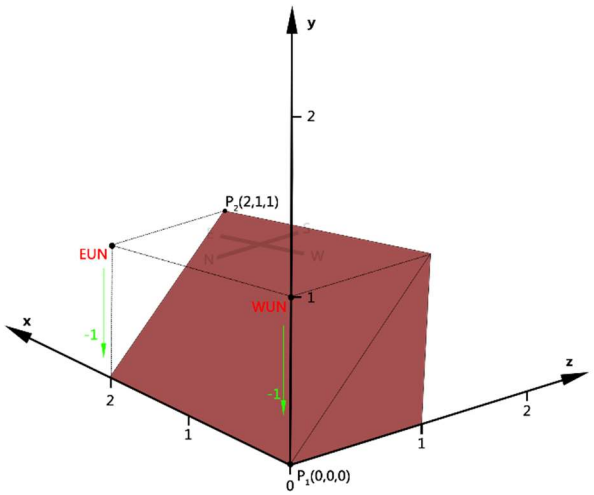
In our example, since we offset the Y values of EUN and EUS, we will need two values in the data to represent these offsets. The order is written from right to left, so we write the offset value for EUN's Y first, followed by EUS's Y. When reading the data, we check which axes have been offset and then read the corresponding values from the data in right-to-left order, assigning 0 to any axes that have not been offset.

These offset values are stored as 32-bit integers, but the offset amount itself only requires 16 bits. Therefore, when reading, we need to split the 32-bit signed integer into two 16-bit parts, each representing a separate offset value.

Thus, each number from the 8th value onward in the array actually represents two offset values.

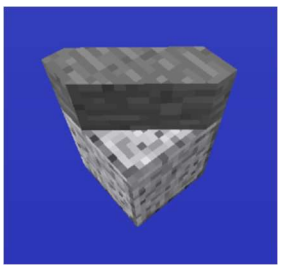
Another more straightforward example is:

```
Tiles info (BLOCK):
    block coord: 15 4 10
    grid: 2
    id: minecraft:littletilestileentity
    boxes count: 1
    Tile info:
        block id: minecraft:stained_hardened_clay:6
        boxes:  [0, 0, 0, 2, 1, 1, -2147475454, -1]
        Angle change state (from -2147475454):
        |FLIPPED|WDS|WDN|WUS|WUN|EDS|EDN|EUS|EUN|
        | EWSNUD|zyx|zyx|zyx|zyx|zyx|zyx|zyx|zyx|
        | 000000|000|000|000|010|000|000|000|010|
        Angle offset (from -1):
        WUN:  y_offset: -1
        EUN:  y_offset: -1
tips:
  P_1(0,0,0) 来自 [0,0,0,2,1,1,...]
  P_2(2,1,1) 来自 [0,0,0,2,1,1,...]
  -1 的二进制表示为 11111111,11111111,11111111,11111111 (32bit)
  WUN y_offset 来自前16位，即 11111111,11111111
  EUN y_offset 来自后16位，即 11111111,11111111
  grid 代表该方块内的小方块精度是 2
```

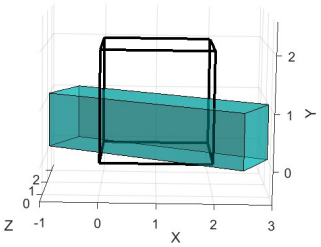


For some other slopes, when you calculate their offsets, they might exceed the boundaries of the block. For example, consider the small block on the right with a precision of 2. The offset values for each corner are as follows:

WDS:	x_offset: -2	z_offset: 2
WDN:	x_offset: 3	z_offset: -2
WUS:	x_offset: -2	z_offset: 2
WUN:	x_offset: 3	z_offset: -2
EDS:	x_offset: -3	z_offset: 2
EDN:	x_offset: 2	z_offset: -2
EUS:	x_offset: -3	z_offset: 2
EUN:	x_offset: 2	z_offset: -2



We can observe that all its corners exceed the boundaries of the block. The actual effect is as follows:



What we see in the game is just the shape after being clipped by the block boundaries: